

# Behavioral UML Modeling

## Sequence Diagrams & Statechart Diagrams

A visual grammar for modeling how software objects behave, communicate, and change state over time.



Department of Computer Science & Engineering



UML 2.5

### BEHAVIORAL DIAGRAMS



#### Sequence Diagram

interaction over time



#### Statechart Diagram

object lifecycle & states

# The Behavioral View of Software

Every system has two complementary faces — what it is made of, and how it acts over time.



## Static Modeling

- Captures structure: classes, objects, attributes, relationships
- Answers "what does the system consist of?"
- Expressed via Class, Object, Component & Deployment diagrams
- Fixed at a point in time — a snapshot



## Behavioral Modeling

- Captures dynamics: interactions, states, events, transitions
- Answers "how does the system behave over time?"
- Expressed via Sequence, Statechart, Activity diagrams
- Reveals order, timing, and change

## Why Behavioral Diagrams Matter

- Validate object collaborations before writing code
- Expose race conditions, invalid states, and missing events early
- Bridge requirements (use cases) to design (class diagrams)

## SDLC Integration

- Analysis: model use-case scenarios as sequences
- Design: refine into statecharts for stateful objects
- Testing: derive test cases directly from transitions

# UML Behavioral Diagrams

Five diagram types capture different dimensions of runtime behavior.

THIS UNIT



## Sequence Diagram

Object interactions  
ordered along a timeline

THIS UNIT



## Statechart Diagram

The states an object  
passes through in its life



## Activity Diagram

Workflow and control flow  
across actions



## Communication Diagram

Interactions organized  
around object links



## Interaction Overview

A control-flow map of  
other interactions

# Sequence Diagram

## Definition

A behavioral diagram that shows how objects interact by exchanging messages, arranged in time order along vertical lifelines.

## Purpose

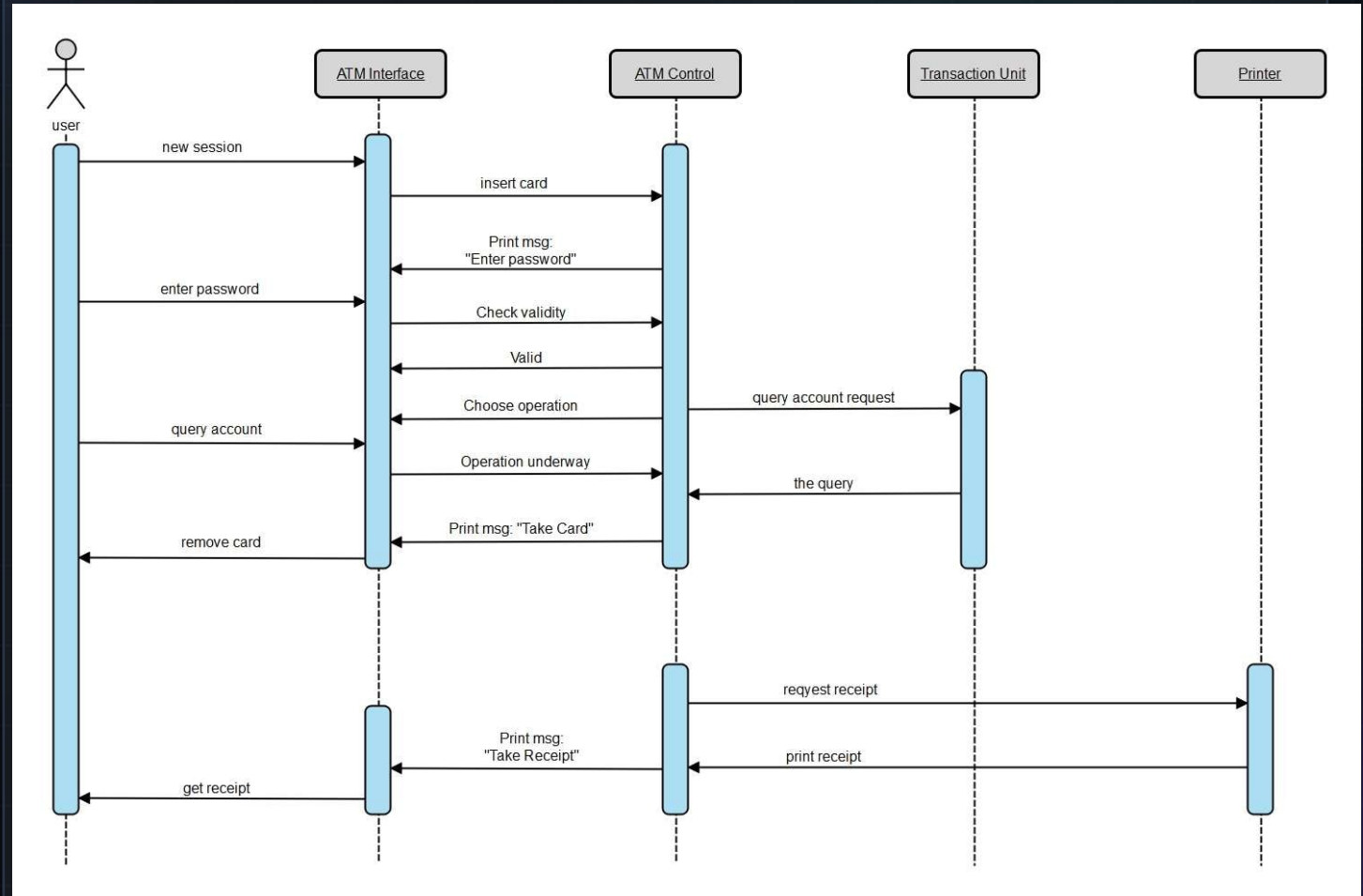
Visualize the exact order of operation calls between objects to validate a scenario before implementation.

## Applications

- API & microservice call flows
- Use-case realization in design
- Debugging distributed transactions

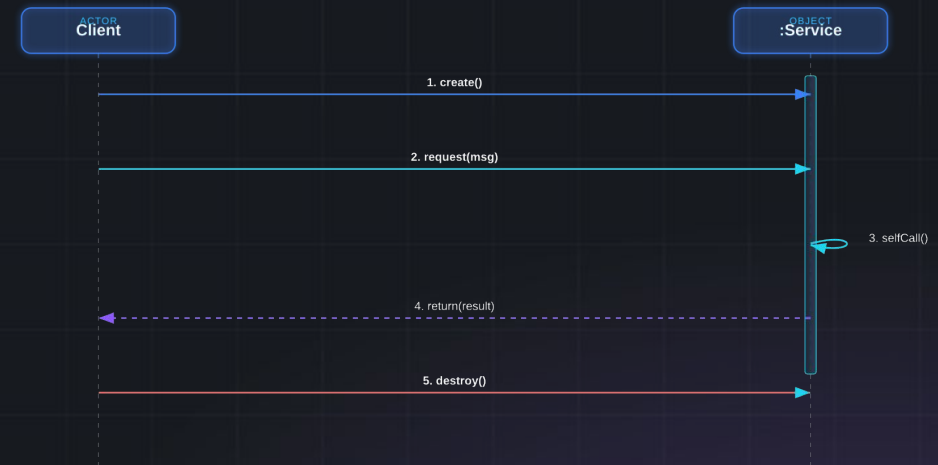
## Advantages

- Makes timing & order explicit
- Easy for non-technical stakeholders to follow
- Directly traceable to test cases



# Components of a Sequence Diagram

- **Actor** — External entity initiating interaction
- **Object / Lifeline** — Participant + its dashed vertical timeline
- **Activation Bar** — Period an object is actively executing
- **Message (Call)** — Solid arrow — synchronous request
- **Return Message** — Dashed arrow — response to a call
- **Self Call** — Looped arrow — object messages itself
- **Create Message** — Arrow to a new object at its birth point
- **Destroy Message** — Arrow ending in an "X" — object lifecycle ends



Annotated notation: create, call, self-call, return & destroy messages

# Case Study: ATM Cash Withdrawal

Customer → ATM → Bank Server →  
Account Database



## 1. Insert Card

Customer inserts card at the ATM



## 2. PIN Verification

ATM sends PIN to Bank Server for auth



## 3. Balance Check

Server checks funds via Account DB



## 4. Withdrawal

Debit request processed & confirmed



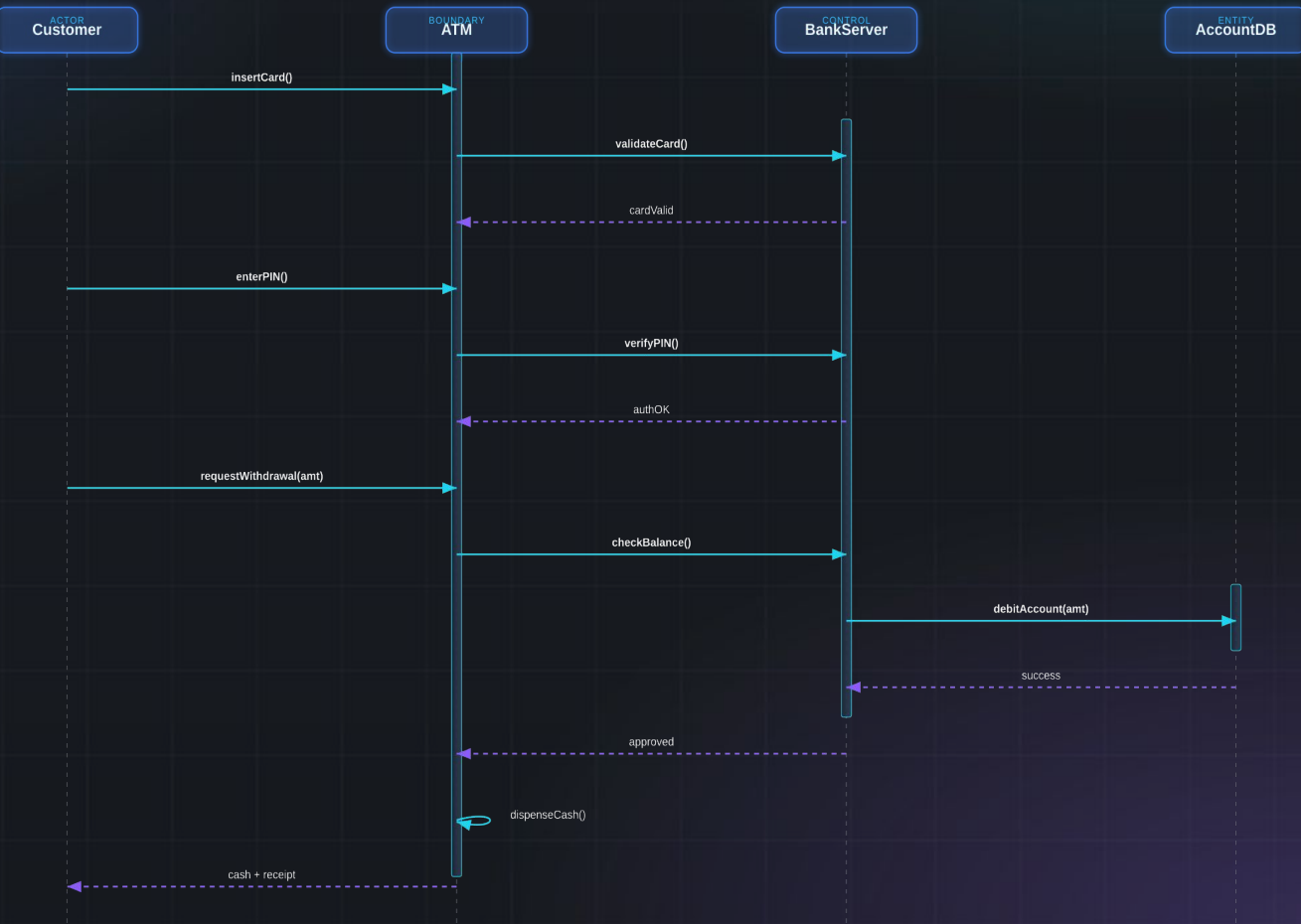
## 5. Cash Dispense

ATM self-call dispenses currency notes



## 6. Receipt

Transaction slip returned to customer

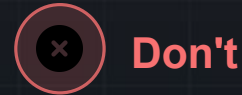


# Sequence Diagram Best Practices



## Do

- Name messages as verbs: `getBalance()`, not `Balance`
- Keep one scenario per diagram — split alternates separately
- Show activation bars only where objects are truly executing
- Order lifelines to match the natural flow, left to right



## Don't

- Cram an entire use case with every alternate path
- Mix abstraction levels — UI clicks next to DB queries
- Omit return messages when a value is actually needed
- Let lifelines cross unnecessarily — reorder actors instead

## Industrial Applications

API contract design

Microservice tracing

Test-case derivation

Legacy system  
reverse-engineering

# Statechart Diagram

## Definition

A finite state machine that models the states an object occupies and the events that trigger transitions between them.

## State Machine

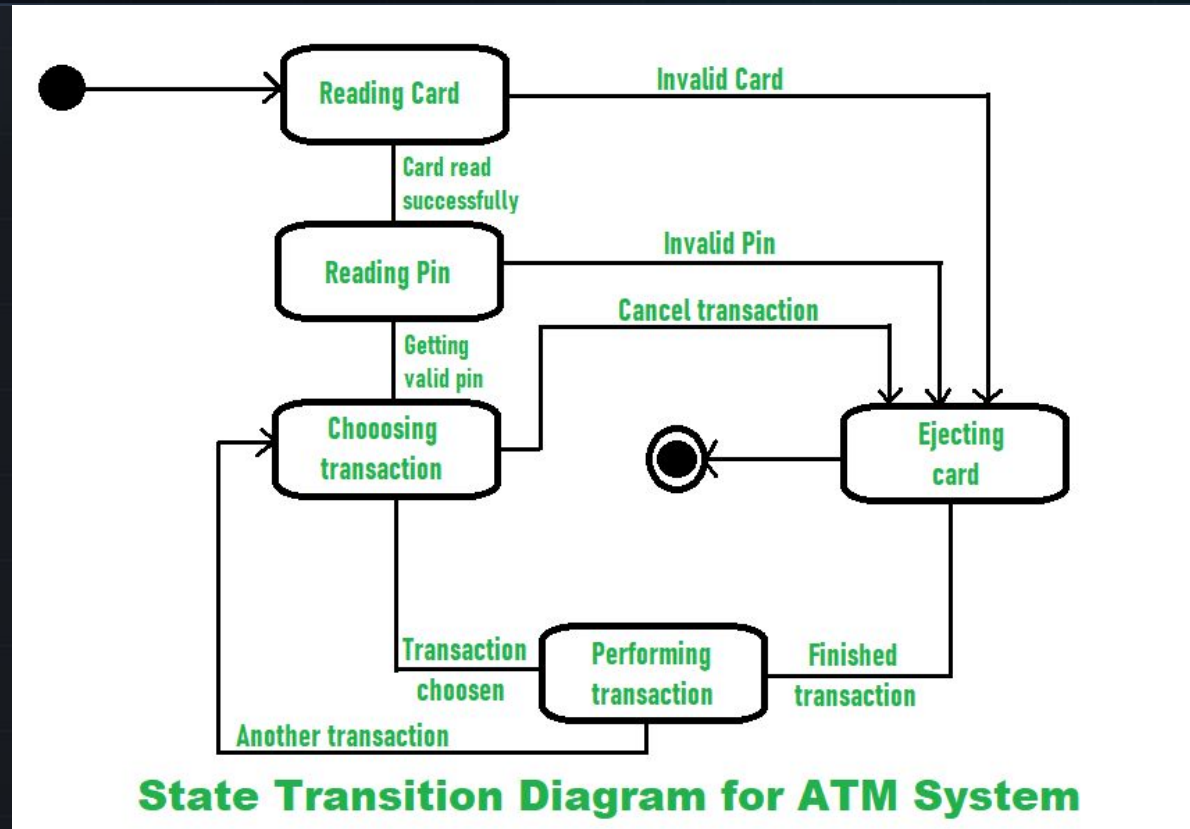
A formal model of finite states plus the rules governing movement from one state to the next.

## Object Lifecycle

Tracks an object from creation (initial state) to destruction (final state) across its entire lifetime.

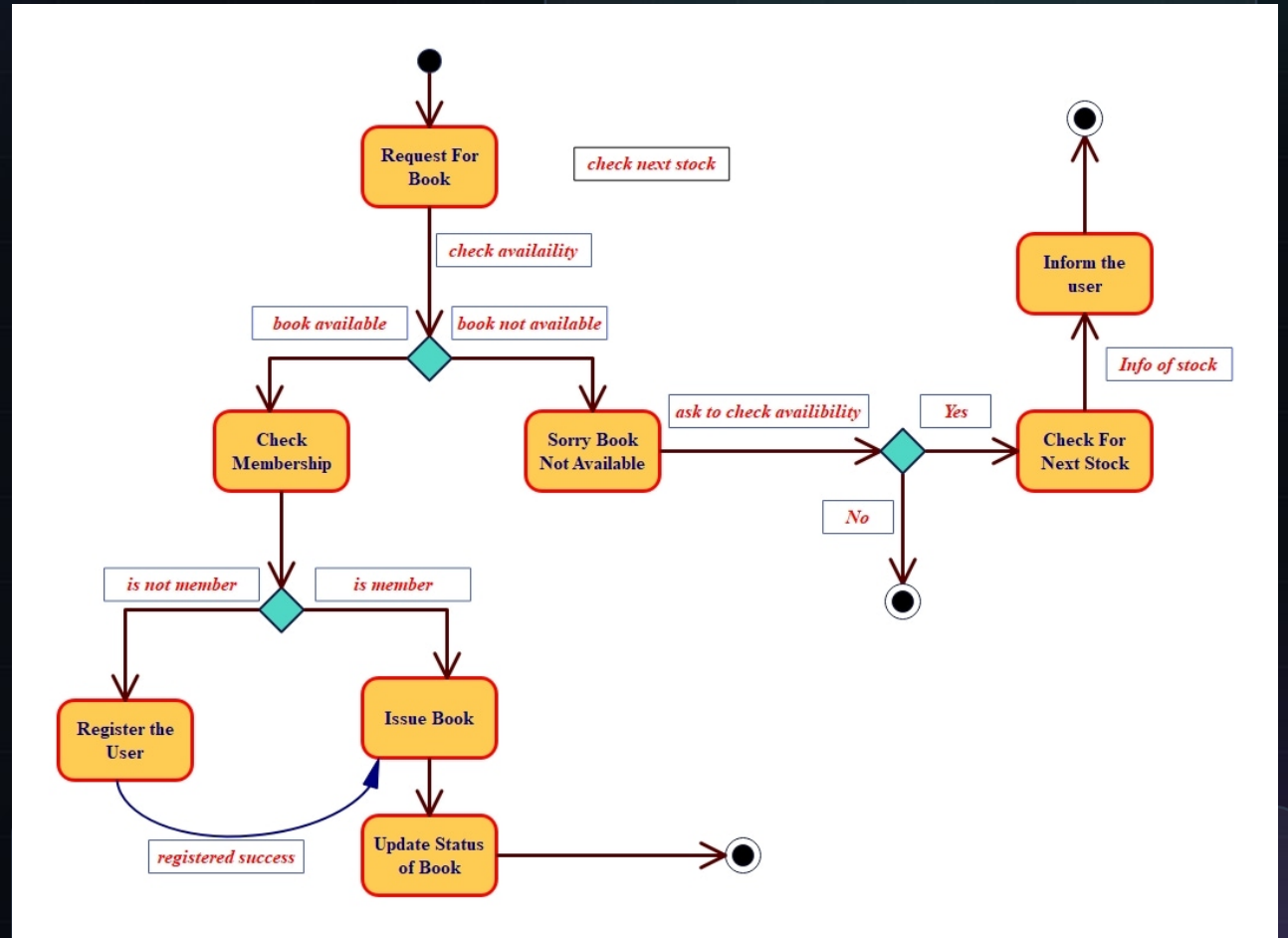
## Events & Transitions

- Event: a trigger such as submit() or timeout()
- Transition: the state-to-state move an event causes



# Components of a Statechart Diagram

- **Initial State** — Solid filled circle — the entry point
- **State** — Rounded rectangle — a condition the object holds
- **Event** — A trigger that may cause a transition
- **Transition** — Arrow between two states, labeled by its event
- **Guard** — Boolean condition in [brackets] gating a transition
- **Action** — Behavior executed on entry, exit, or during transition
- **Composite State** — A state containing nested sub-states
- **Final State** — Circle with a ring — the terminal state



- Transition label syntax: event [guard] / action
- A state may have entry/, exit/, and do/ actions
- Composite states group related sub-states for clarity

# Sequence Diagram vs. Statechart Diagram

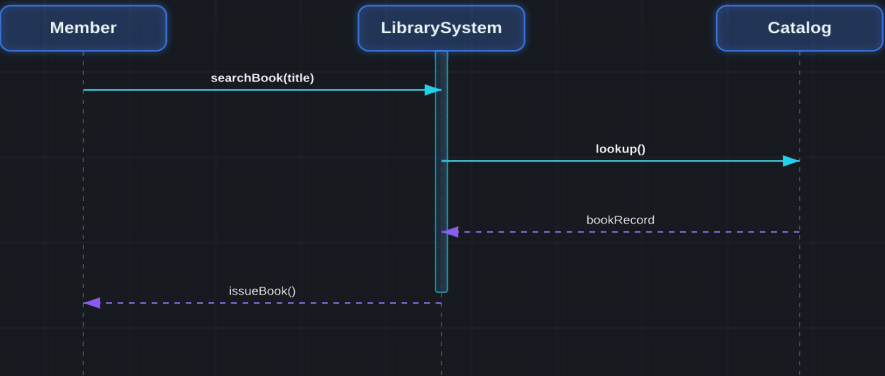
| Aspect          | Sequence Diagram                                | Statechart Diagram                              |
|-----------------|-------------------------------------------------|-------------------------------------------------|
| Purpose         | Shows how objects collaborate to achieve a task | Shows how a single object changes over its life |
| Focus           | Message exchange & interaction order            | States, events, and transitions                 |
| Objects         | Multiple interacting objects                    | One object (or class) at a time                 |
| Lifecycle       | Implicit — via create/destroy messages          | Explicit — the diagram <i>is</i> the lifecycle  |
| Time Dependency | Strongly time-ordered, top to bottom            | Event-driven, not strictly time-ordered         |
| Communication   | Explicit messages between lifelines             | No inter-object messages shown                  |
| Applications    | API design, use-case realization                | Protocol design, UI/order/device lifecycles     |
| Advantages      | Clear interaction order & timing                | Clear valid/invalid state coverage              |
| Limitations     | Hard to show many alternate paths               | Doesn't show collaboration between objects      |

# Library Management System

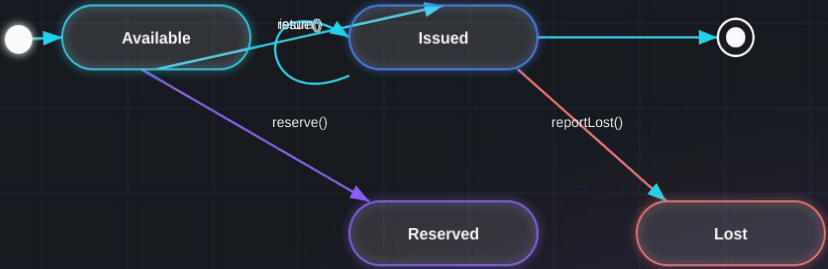


Scenario: a member searches the catalog and issues a book; the same book independently tracks its own Available → Issued → Returned lifecycle.

Sequence Diagram



Statechart Diagram



**Relationship:** the `issueBook()` message in the sequence diagram is exactly the event that fires the `Available → Issued` transition in the statechart — the two views describe the same event from two angles.

## WHERE IT IS USED

# Industrial Applications

Behavioral UML diagrams drive design and validation across virtually every software-intensive industry.



### Banking

Transaction sequencing & account state validation



### Healthcare

Patient record workflows & device states



### Railway Reservation

Booking interactions & seat-status lifecycle



### ERP Systems

Cross-module process orchestration



### IoT

Sensor event-driven state machines



### Smart Manufacturing

Machine control sequencing on the shop floor



### E-Commerce

Checkout flows & order lifecycle



### Mobile Applications

Screen/session state & API call sequencing

# Laboratory Exercise



## Problem Statement

Model a Railway Ticket Reservation System: design a sequence diagram for the "Book Ticket" scenario and a statechart diagram for the lifecycle of a single ticket.



## Deliverables & Submission

- One sequence diagram (draw.io / StarUML / hand-drawn, scanned)
- One statechart diagram for the Ticket object
- A one-paragraph scenario description
- Submit as a single PDF via the LMS portal, named RollNo\_Lab4.pdf

## Step-by-Step Procedure

- 1 Identify actors & objects: Passenger, ReservationSystem, PaymentGateway, TicketDB
- 2 List messages in order: searchTrain(), selectSeat(), makePayment(), confirmBooking()
- 3 Draw lifelines and place activation bars where each object is busy
- 4 Identify the Ticket object's states: Available, Reserved, Confirmed, Cancelled
- 5 Draw transitions with their triggering events and any guards
- 6 Cross-check: every sequence message should map to a valid state transition

*Evaluation Rubric: Correctness (40%) · Notation Accuracy (30%) · Clarity (20%) · Timeliness (10%)*

# Summary & Viva Questions



Sequence diagrams order messages between objects over time



Statecharts model an object's lifecycle through states & events



Choose the diagram that matches the question you are answering



Both diagrams complement class diagrams during design

## 10 Viva Questions

1. What is the difference between a sequence and a collaboration/communication diagram?
2. How does an activation bar differ from a lifeline?
3. What does a dashed arrow represent in a sequence diagram?
4. Define "guard condition" with an example.
5. What is a composite state, and why is it useful?
6. How would you represent object destruction in UML?
7. What triggers a self-call, and when would you use one?
8. Differentiate between an event and an action in a statechart.
9. Can a statechart have more than one final state? Explain.
10. How do sequence diagrams support test-case design?

## Thank You

Questions & Discussion

